



ACCRA TECHNICAL UNIVERSITY, GHANA
FACULTY OF APPLIED SCIENCES
DEPARTMENT OF COMPUTER SCIENCE
BTECH COMPUTER SCIENCE - YEAR II
COURSEWORK ASSIGNMENT 1

BCP 210: DATA STRUCTURES AND ALGORITHMS I
ACADEMIC YEAR: 2025/2026

STUDENT NAME: _____

INDEX NUMBER: _____

INSTRUCTIONS TO STUDENTS:

1. Answer ALL questions.
2. Implement all algorithms in Python (version 3.8 or above).
3. **GitHub Submission Steps:**
 - (a) Go to the starter repository: <https://github.com/atu-dsa-2025/dsa-assignment-1-template>
 - (b) Click the green "Use this template" button → select "Create a new repository".
 - (c) Set the repository visibility to **Private** and name it **dsa-assignment-1-INDEXNUMBER**.
 - (d) Work on your solutions locally, commit, and push your python files (.py) and report (.pdf) to your repository.
 - (e) **Submit by Transferring Ownership:** When finished, go to your repository's **Settings** → scroll to the bottom **Danger Zone** → click **Transfer ownership** → and transfer it to the class organization: **atu-dsa-2025**.
4. **Submission Verification:** Your submission date is verified strictly by the ****GitHub server push timestamp****. Local commit timestamps will not be accepted as proof of on-time submission.
5. **Late Policy:** Submissions pushed after the deadline will incur a 10% penalty per 24-hour delay.

1 Assignment 1: Arrays (45 Marks)

Topic: University Course Registration Analytics

Background

Accra Technical University stores student registrations for a specific course in a Python list representing the sequence of entries as they arrive.

```

1 registrations = [
2     1023, 1050, 1023, 1102,
3     1050, 1023, 1201, 1102,
4     1300, 1023
5 ]

```

Each integer in the list represents a unique Student ID.

Problem Statement

Write a Python program that implements efficient algorithms to perform the following operations:

1. **Most Frequent Registrant (8 Marks):** Find the student ID that appears the most times in the registration list.
2. **Ordered Deduplication (10 Marks):** Remove all duplicate student IDs from the list while preserving the exact order of their first appearance.
3. **First Unique Student (9 Marks):** Identify the first student ID in the registration list that appears exactly once.
4. **Contiguous Subarray Sum (10 Marks):** Determine whether there exists a contiguous subarray of registrations whose sum equals a given target value T (e.g., check if a sequence of IDs sum to a target).
5. **Complexity Analysis (8 Marks):** Analyze and state the worst-case time and space complexities for each of your four solutions using Big-O notation.

Expected Output Format

Your script should print the results in the following format:

```

1 Most frequent student: 1023
2 Unique registrations: [1023, 1050, 1102, 1201, 1300]
3 First non-repeated student: 1201
4 Subarray with target sum exists: True

```

2 Assignment 2: Linked Lists (45 Marks)

Topic: University Helpdesk Ticket Queue

Background

The university IT helpdesk maintains support tickets in the order they arrive. To support dynamic additions and updates without the memory overhead of contiguous array shifts, the department wants to use a Singly Linked List. A ticket node contains:

- `ticket_id` (Integer)
- `student_name` (String)
- `issue` (String)

Problem Statement

Implement a singly linked list class (`TicketQueue`) from scratch that supports the following operations:

1. **Enqueue Ticket (8 Marks):** Add a new ticket to the end of the queue.
2. **Priority Insert (10 Marks):** Insert a new priority ticket immediately after a specified ticket ID.
3. **Resolve Ticket (9 Marks):** Delete a resolved ticket from the queue given its ticket ID.
4. **Find Middle Ticket (8 Marks):** Find the middle ticket of the queue in a single traversal pass (using the fast/slow pointer method).
5. **Reverse Queue (10 Marks):** Reverse the queue in-place (manipulating pointers without copying data).
6. **Display Queue (4 Marks):** Traversal method to print all ticket IDs and details.
7. **Complexity Analysis (6 Marks):** State the worst-case time complexity of each operation.

Starter Classes

Use the following starter code to begin your implementation:

```
1 class Ticket:
2     def __init__(self, ticket_id, student_name, issue):
3         self.ticket_id = ticket_id
4         self.student_name = student_name
5         self.issue = issue
6         self.next = None
7
8 class TicketQueue:
9     def __init__(self):
10         self.head = None
```

3 Bonus Challenge (10 Marks)

The university IT department currently stores tickets in a singly linked list. If the helpdesk workload shifts such that they perform thousands of searches per day but only a few insertions and deletions, would a Python list (contiguous array) be more appropriate? Justify your answer thoroughly, referencing:

- Big-O time complexity of search operations.
- CPU cache locality and hardware memory access patterns.
- Memory allocation overhead (contiguous arrays vs. node pointer chasing).

4 Marking Scheme

The assignment is graded out of 100 marks. The grade distribution is defined as follows:

Table 1: Coursework Assignment 1 Grade Rubric

Component	Description	Marks
Correct Implementation	Algorithms produce correct outputs under standard conditions.	40
Edge Case Handling	Handles empty lists, single elements, unfound elements, and large inputs gracefully without crashing.	20
Complexity Analysis	Clear explanation of runtime and spatial scaling bounds using Big-O notation.	15
Code Quality	Structured modular code, descriptive variable names, and helpful inline comments.	15
Testing	Robust test cases verifying boundary values and typical lists.	10
Total		100